



SAIR

Spatial AI & Robotics Lab

CSE 473/573

L9: ALIGNMENT & FITTING

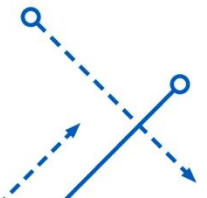
Chen Wang

Spatial AI & Robotics Lab

Department of Computer Science and Engineering

What are Alignment and Fitting?

- Alignment
 - Find the parameters of a transformation that best aligns matched points
- Fitting
 - Find the parameters of a model that best fit the data



Motivation

- Getting the whole picture
 - Typical camera: $50^\circ \times 35^\circ$



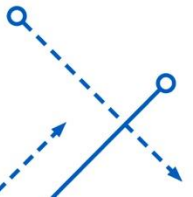
Motivation

- Getting the whole picture
 - Typical camera: $50^\circ \times 35^\circ$
 - Human Vision: $176^\circ \times 135^\circ$

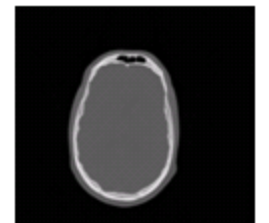
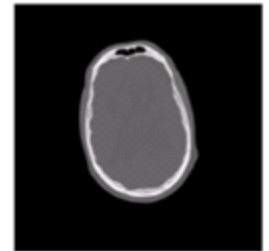
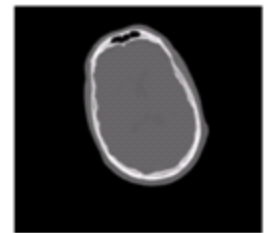
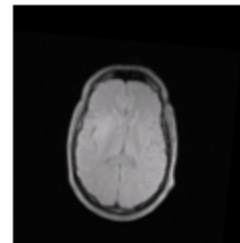
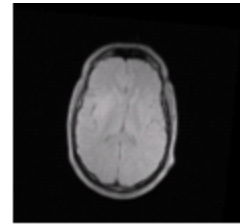
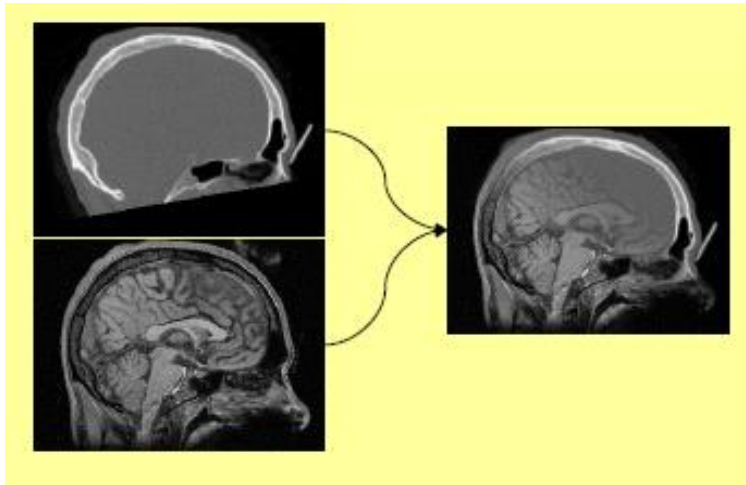


Motivation

- Getting the whole picture
 - Typical camera: $50^\circ \times 35^\circ$
 - Human Vision: $176^\circ \times 135^\circ$



Motivation: Medical image registration



Alignment

- Homography
- Rotational Panoramas
- RANSAC
- Global alignment
- Interpolation (next lecture)
- Blending (next lecture)



(a)



(b)



Motion models

- What happens when we take two images with a camera and try to align them?
- translation?
- rotation?
- scale?
- affine?
- perspective?



Panoramas

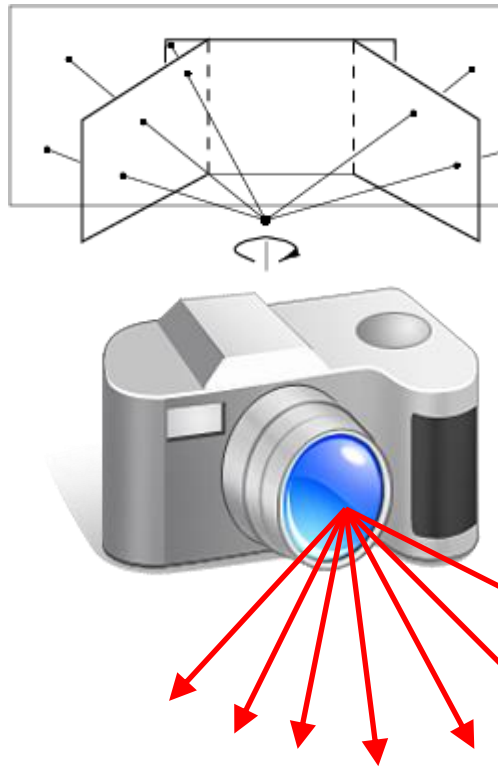


image from S. Seitz

Obtain a wide angle view by combining multiple images.

How to stitch together a panorama?

- Basic Procedure

- Take a sequence of images from the same position
 - Rotate the camera about its **optical center**.
 - Compute transformation between 2nd and 1st image
 - Transform the 2nd image to overlap with the 1st
 - Blend the two together to create a mosaic
 - If there are more images, repeat.
- Why should this work at all?
 - Why do we rotate w.r.t. optical center?

Correspondence

- Allows us to map image back to some real space

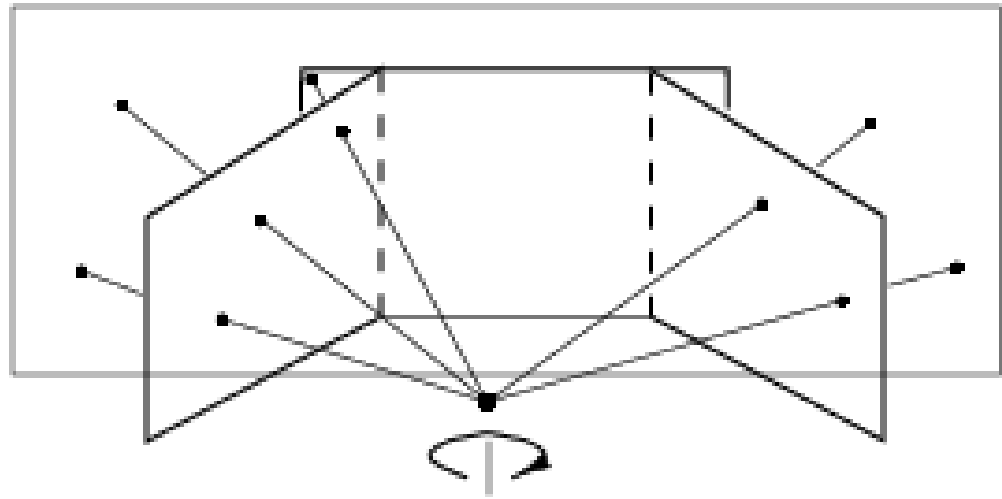
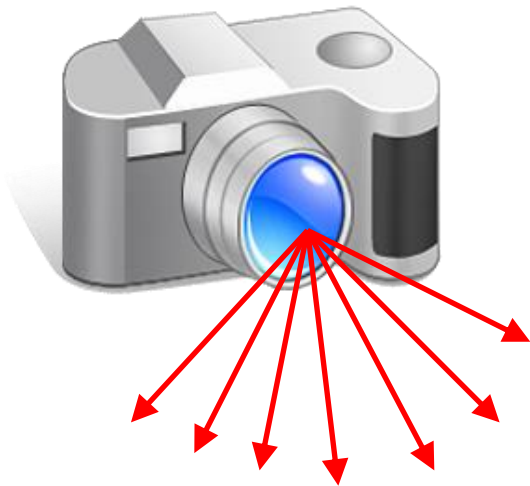
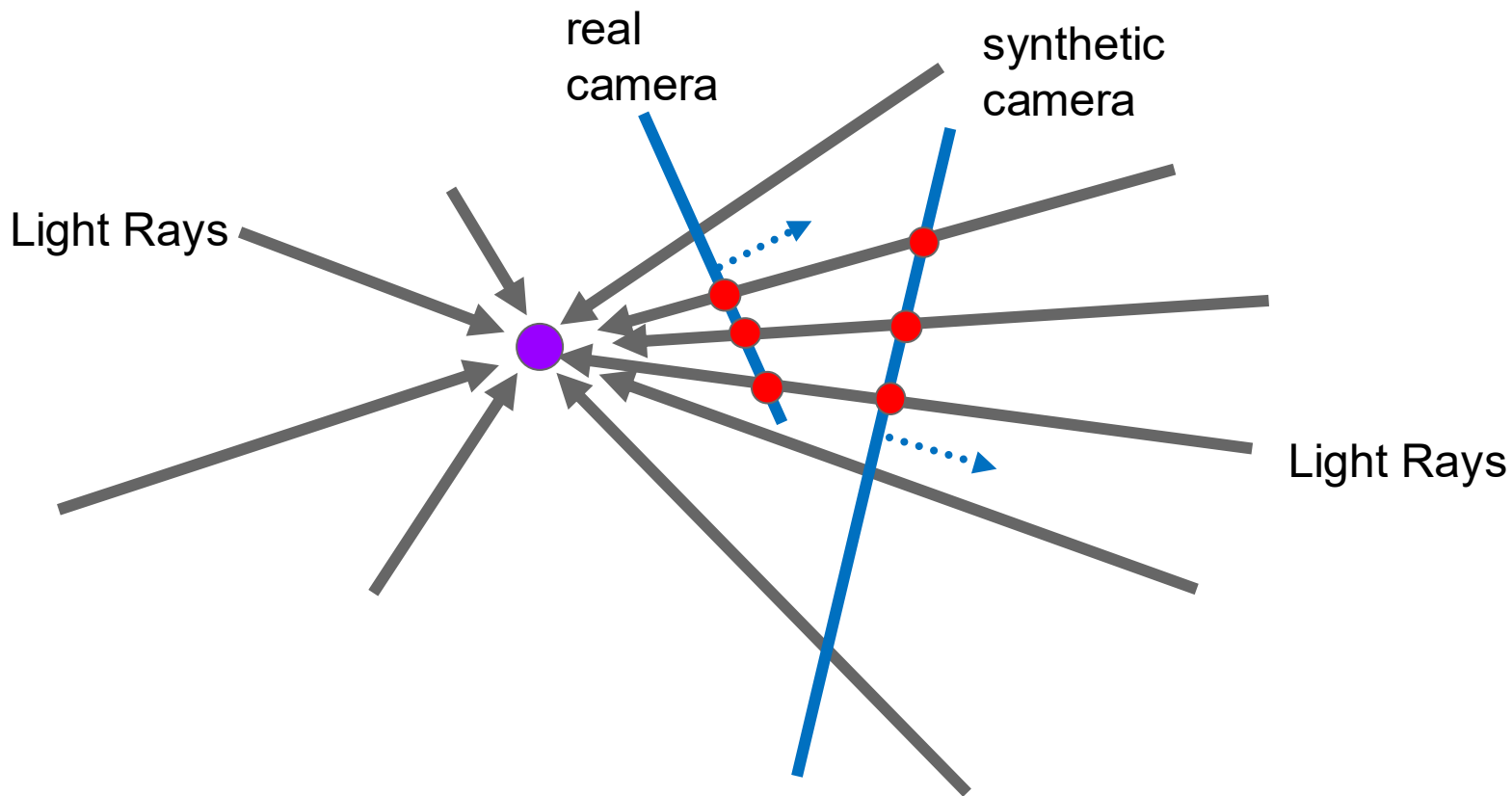


image from S. Seitz

Panoramas: generating synthetic views



Can generate any synthetic camera view
as long as it has **the same center of projection!**

Recap: 2D coordinate transformations

- translation: $\mathbf{x}' = \mathbf{x} + \mathbf{t}$ $\mathbf{x} = (x, y)$
- rotation: $\mathbf{x}' = \mathbf{R} \mathbf{x} + \mathbf{t}$
- similarity: $\mathbf{x}' = s \mathbf{R} \mathbf{x} + \mathbf{t}$
- affine: $\mathbf{x}' = \mathbf{A} \mathbf{x} + \mathbf{t}$
- perspective: $\underline{\mathbf{x}}' \cong \mathbf{H} \underline{\mathbf{x}}$ $\underline{\mathbf{x}} = (x, y, 1)$
($\underline{\mathbf{x}}$ is a *homogeneous* coordinate)
- These all form a nested *group* (closed w/ inv.)

Recap: Basic 2D Transformations

- Basic 2D transformations as 3x3 matrices

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Translate

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Scale

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos \Theta & -\sin \Theta & 0 \\ \sin \Theta & \cos \Theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

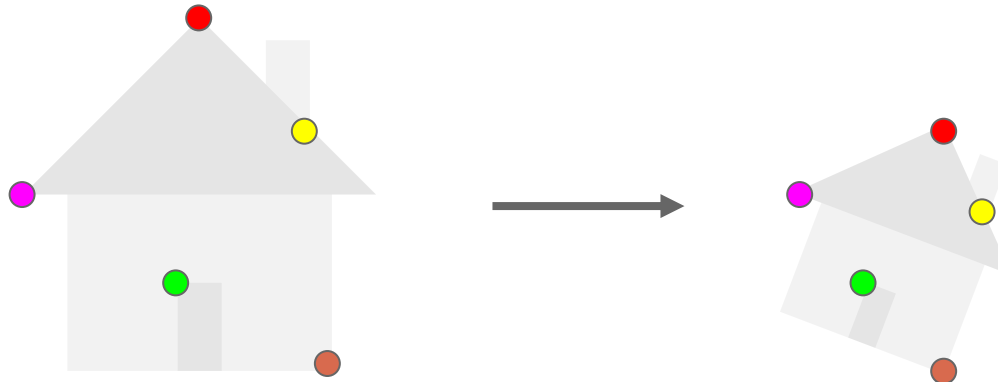
Rotate

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & sh_x & 0 \\ sh_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Shear (Skew)

Image alignment

- Two broad approaches:
 - Direct (pixel-based) alignment
 - Search for alignment where most pixels agree
 - Feature-based alignment
 - Search for alignment where *extracted features* agree
 - Can be verified using pixel-based alignment



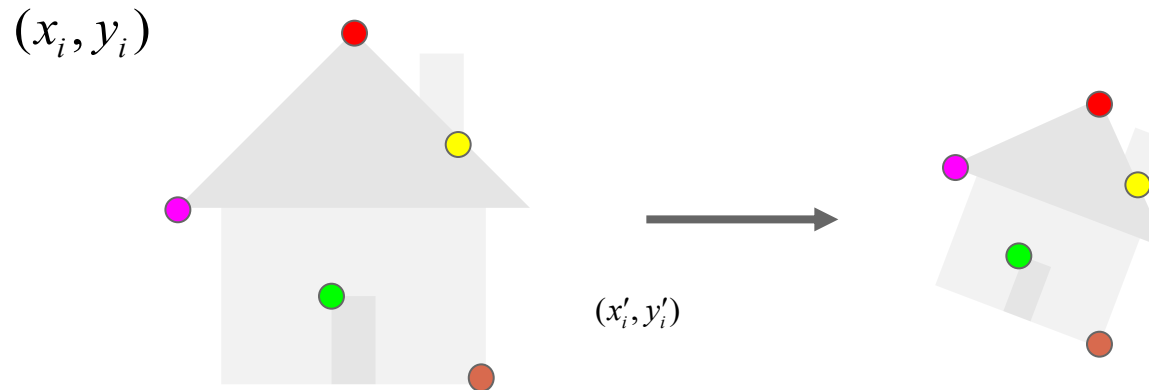
Fitting an affine transformation



Affine model approximates perspective projection of planar objects.

Fitting an affine transformation

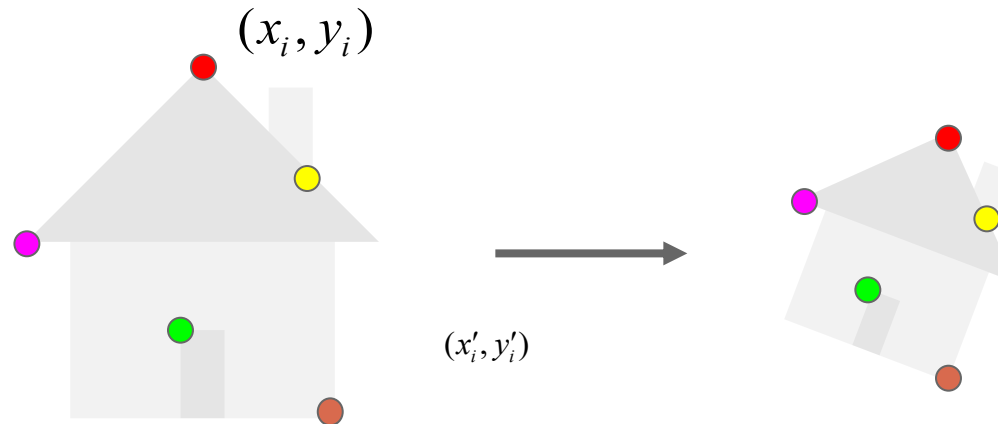
- Assuming we know the correspondences, how do we get the transformation?



$$\begin{bmatrix} x'_i \\ y'_i \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix}$$

Fitting an affine transformation

- Assuming we know the correspondences, how do we get the transformation?



$$\begin{bmatrix} x'_i \\ y'_i \end{bmatrix} = \begin{bmatrix} m_1 & m_2 \\ m_3 & m_4 \end{bmatrix} \begin{bmatrix} x_i \\ y_i \end{bmatrix} + \begin{bmatrix} t_1 \\ t_2 \end{bmatrix}$$

$$\begin{bmatrix}$$

$$\begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix}$$

Fitting an affine transformation

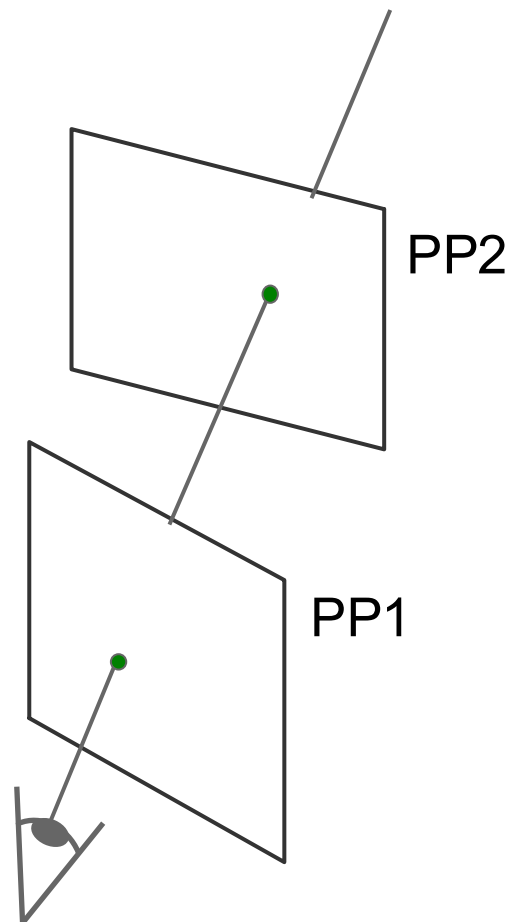
$$\begin{bmatrix} \dots & & & & & \\ x_i & y_i & 0 & 0 & 1 & 0 \\ 0 & 0 & x_i & y_i & 0 & 1 \\ \dots & & & & & \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ t_1 \\ t_2 \end{bmatrix} = \begin{bmatrix} \dots \\ x'_i \\ y'_i \\ \dots \end{bmatrix}$$

- How many matches (correspondence pairs) do we need to solve for the transformation parameters?
- Once we have solved for the parameters, how do we compute the coordinates of the corresponding point for (x_{new}, y_{new}) ?

Homography

- Take as a 2D **image warp** using projective transform.
- A **projective transform** is a mapping between any two PPs with the same center of projection
 - parallel lines aren't preserved.
 - but straight lines are preserved.
 - rectangle should map to arbitrary quadrilateral
- Called **Homography**

$$\begin{bmatrix} wx' \\ wy' \\ w \\ \mathbf{p}' \end{bmatrix} = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \\ \mathbf{H} \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \\ \mathbf{p} \end{bmatrix}$$



Solving for homographies

(x, y)



$$\begin{pmatrix} wx'/w & wy'/w \end{pmatrix} = (x', y')$$

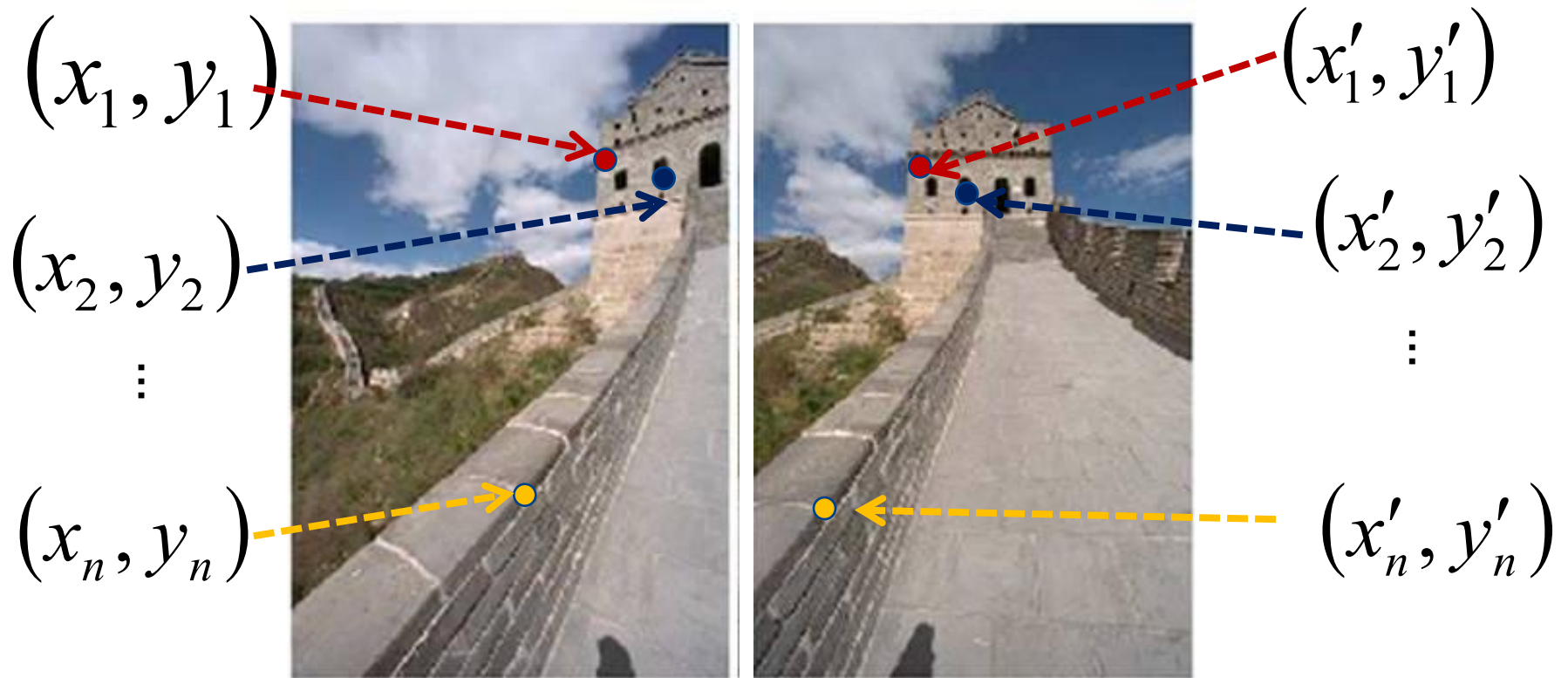
To **apply** a given homography $\mathbf{H}$

- Compute $\mathbf{p}' = \mathbf{H}\mathbf{p}$ (regular matrix multiply)
- Convert $\mathbf{p}'$ from homogeneous to image coordinates

$$\begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

$\mathbf{p}'$ $\mathbf{H}$ $\mathbf{p}$

Solving for homographies



To **compute** the homography given pairs of corresponding points, we need to set up an equation where the parameters of **H** are the unknowns...

Solving for homographies

$$\mathbf{p}' = \mathbf{H}\mathbf{p} \quad \begin{bmatrix} wx' \\ wy' \\ w \end{bmatrix} = \begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

- Can set scale factor $i = 1$ or $\|\mathbf{H}\| = 1$. So, there are 8 unknowns.
- Set up a system of linear equations:

$$\bullet \mathbf{A}\mathbf{h} = \mathbf{b}$$

where vector of unknowns $\mathbf{h} = [a, b, c, d, e, f, g, h]^T$

- Need at least 8 equations (4 points), but the more the better...
- Solve for $\mathbf{H}$. If over-constrained, solve using least-squares:

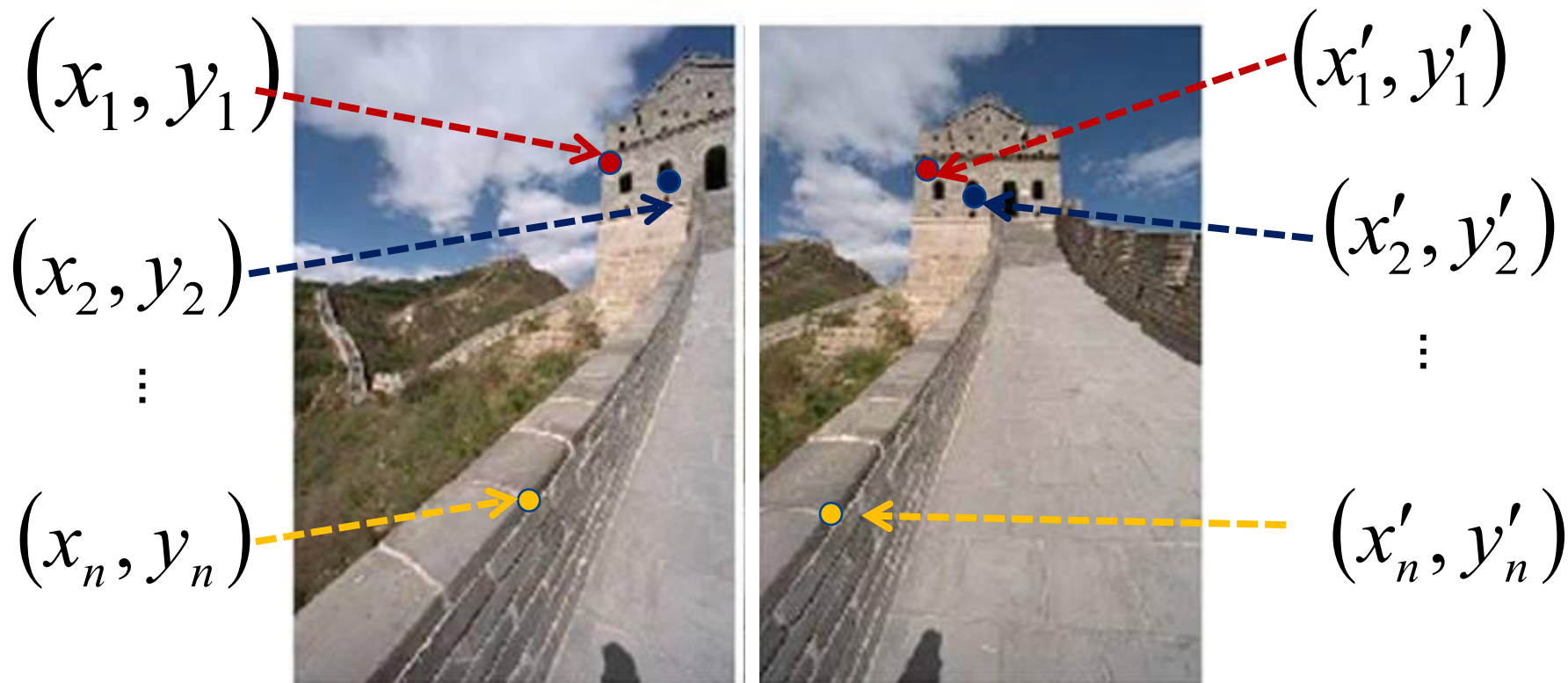
$$\min \|\mathbf{A}\mathbf{h} - \mathbf{b}\|^2$$

$$\mathbf{h} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$$

Proof of least squares

- $F(h) = ||Ah - b||^2 = (Ah - b)^T (Ah - b)$
- $F(h) = h^T A^T Ah - h^T A^T b - b^T Ah + b^T b$
- $\frac{\partial}{\partial h} F(h) = 2A^T Ah - A^T b - (b^T A)^T$
- Setting derivative to 0: $\frac{\partial}{\partial h} F(h) = 0$
- $A^T Ah = A^T b$
- $h = (A^T A)^{-1} A^T b$

Recap: Alignment



What if there are incorrect matches?

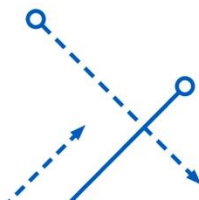
- How can we remove outliers?

Solutions?

- Least Squares Fit is Sensitive to outliers
- Hypothesize and Test
 - Try out as many lines as we want
 - Keep the best lines
 - But which are the best?

RANSAC

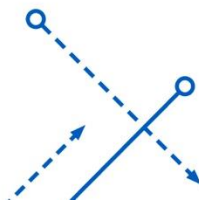
- **RAN**dom **S**ample **C**onsensus
 - An iterative method for estimating a mathematical model from a data set that contains outliers.
- Motivation: we want to avoid the impact of outliers, so let's look for “inliers”, and use those only.
- Idea: if an outlier is chosen to compute the current model, then the model won't have much support from rest of the points.



RANSAC

RANSAC loop:

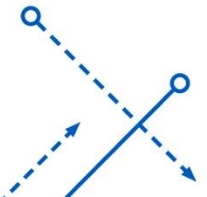
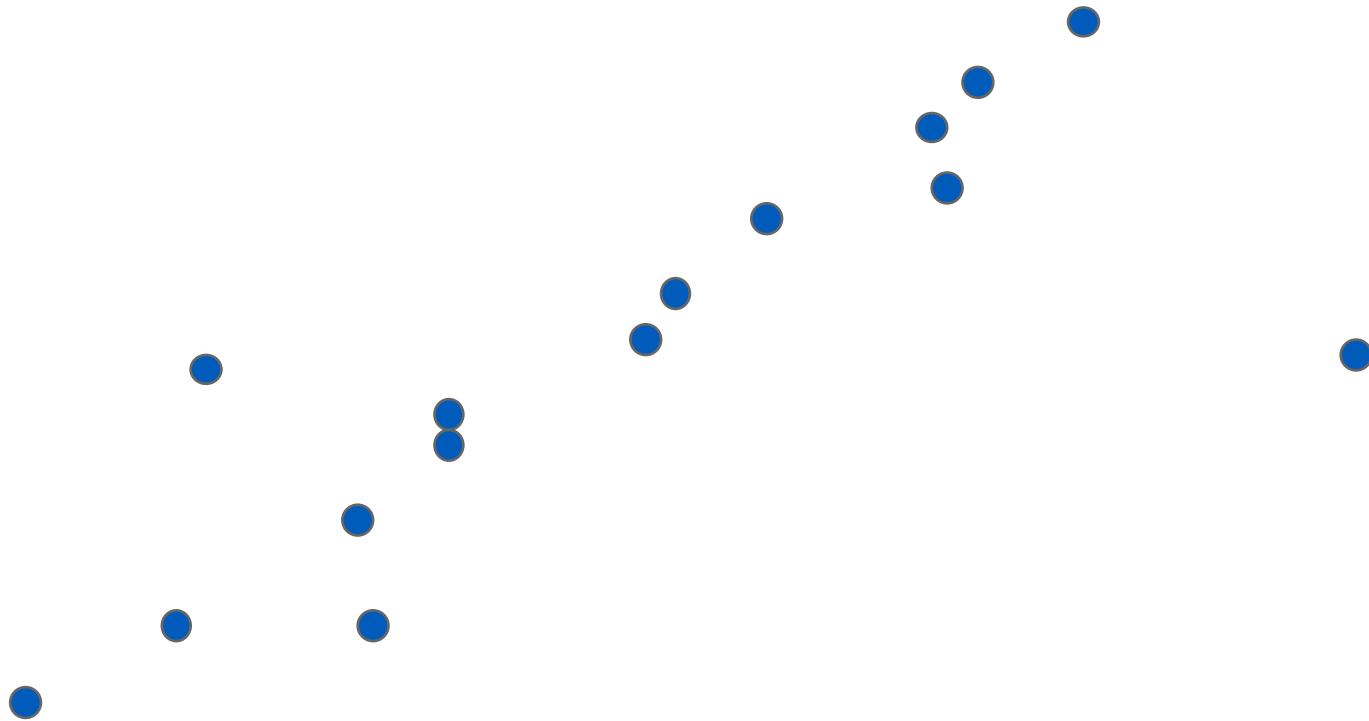
1. Randomly select a **seed group** of points on which to base transformation estimate (e.g., a group of matches)
 2. Compute transformation (**model**) from seed group
 3. Find **inliers** to this transformation
 4. If the number of inliers is sufficiently large, re-compute least-squares estimate on all inliers.
- Keep the model with the **largest number of inliers**.



RANSAC Line Fitting Example

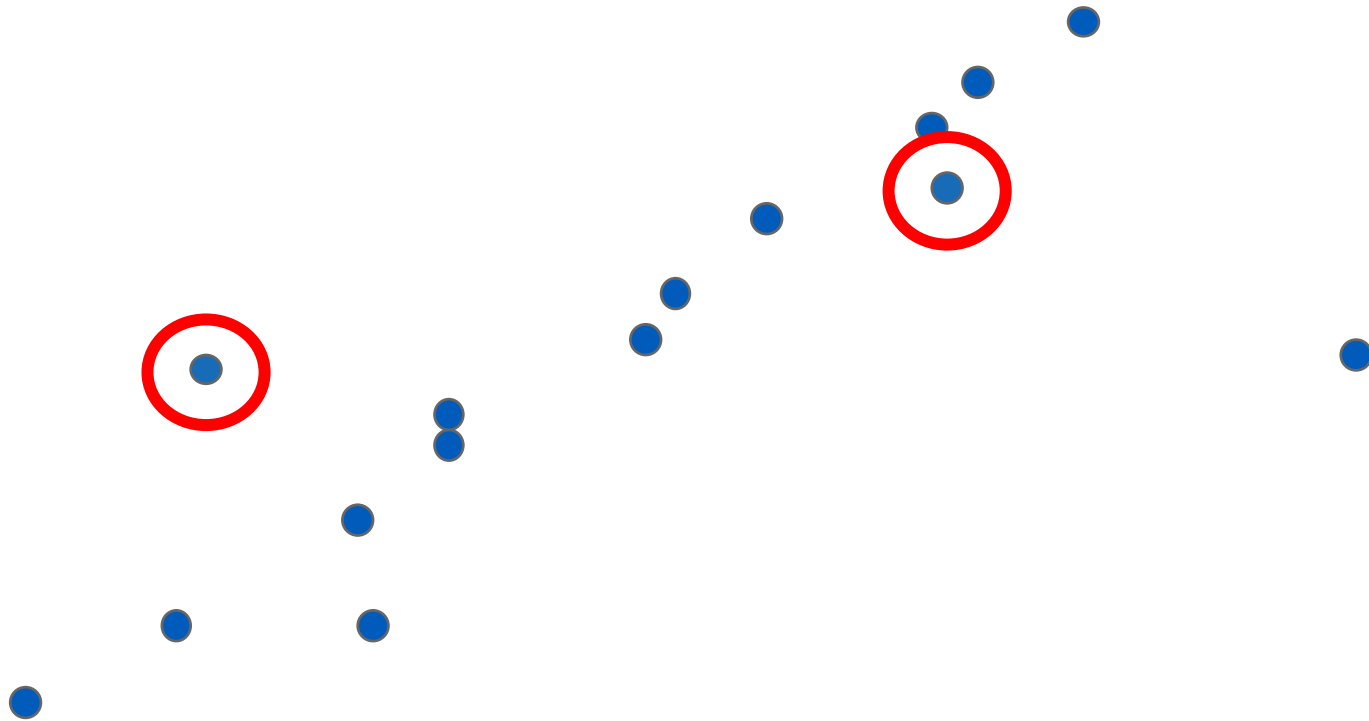
Task:

Estimate best line



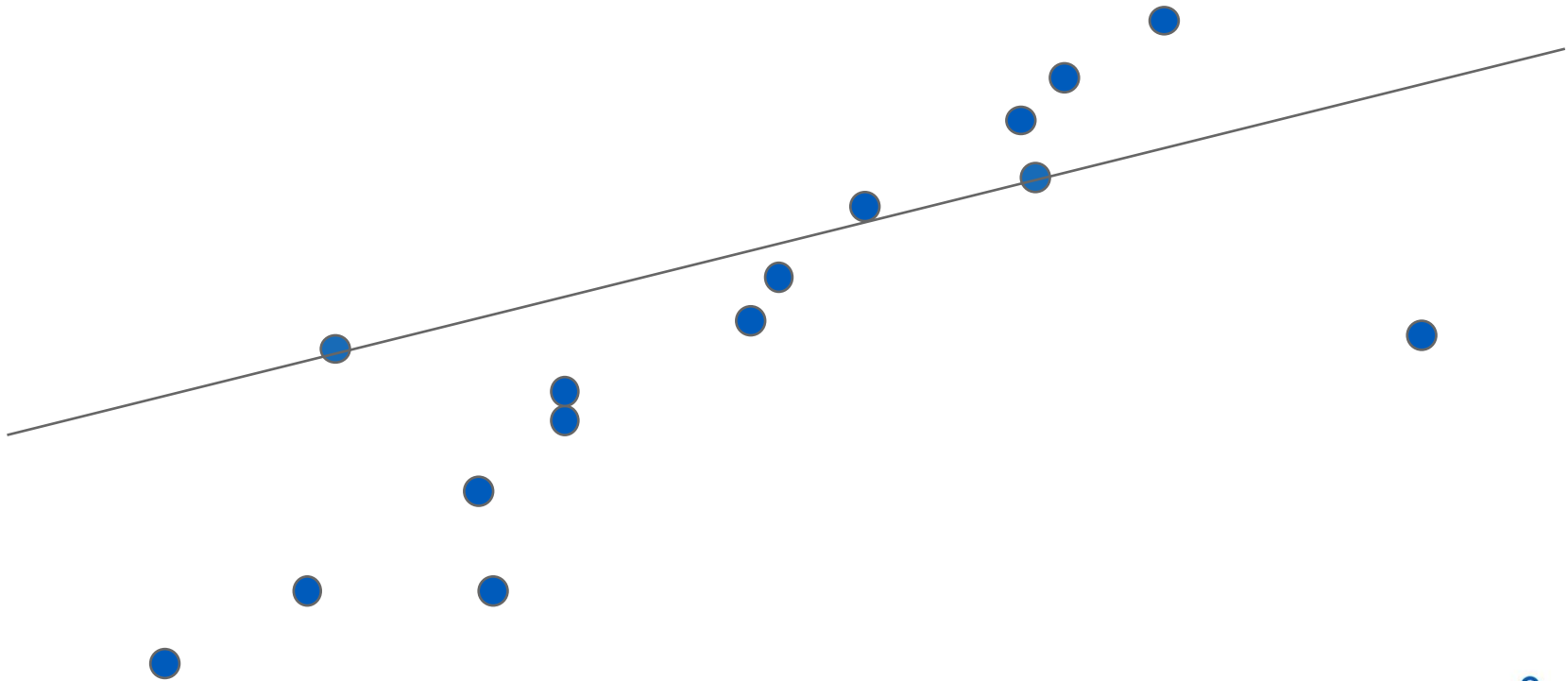
RANSAC Line Fitting Example

Sample two points



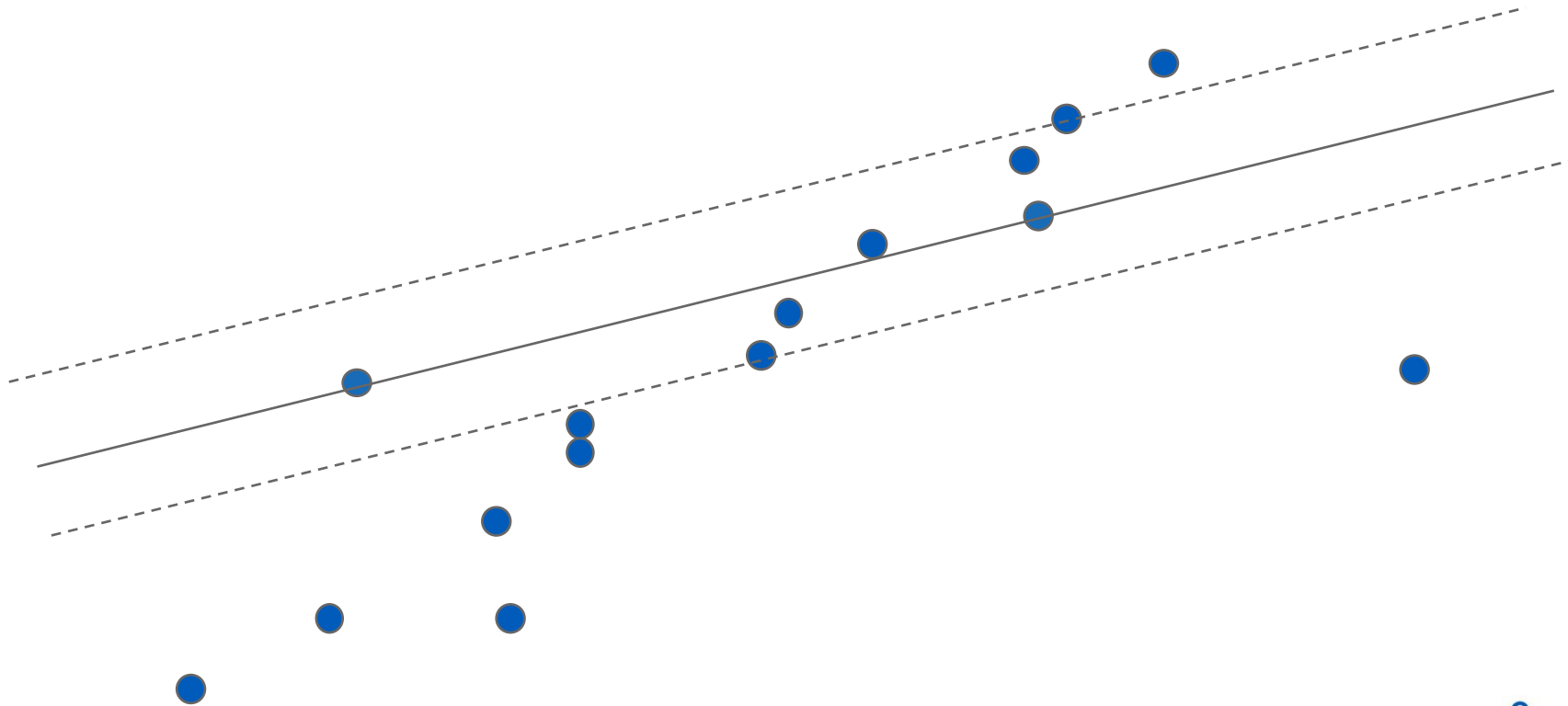
RANSAC Line Fitting Example

Fit Line



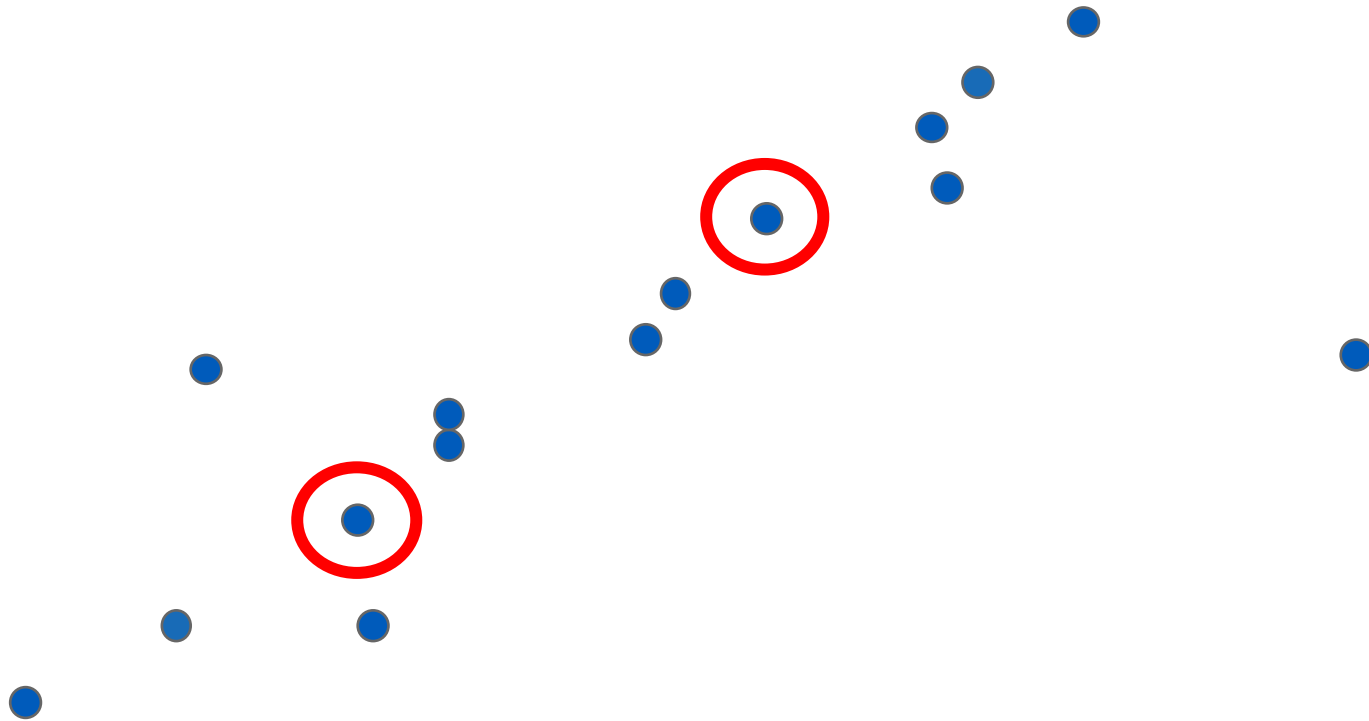
RANSAC Line Fitting Example

Total number of points within a threshold of line.



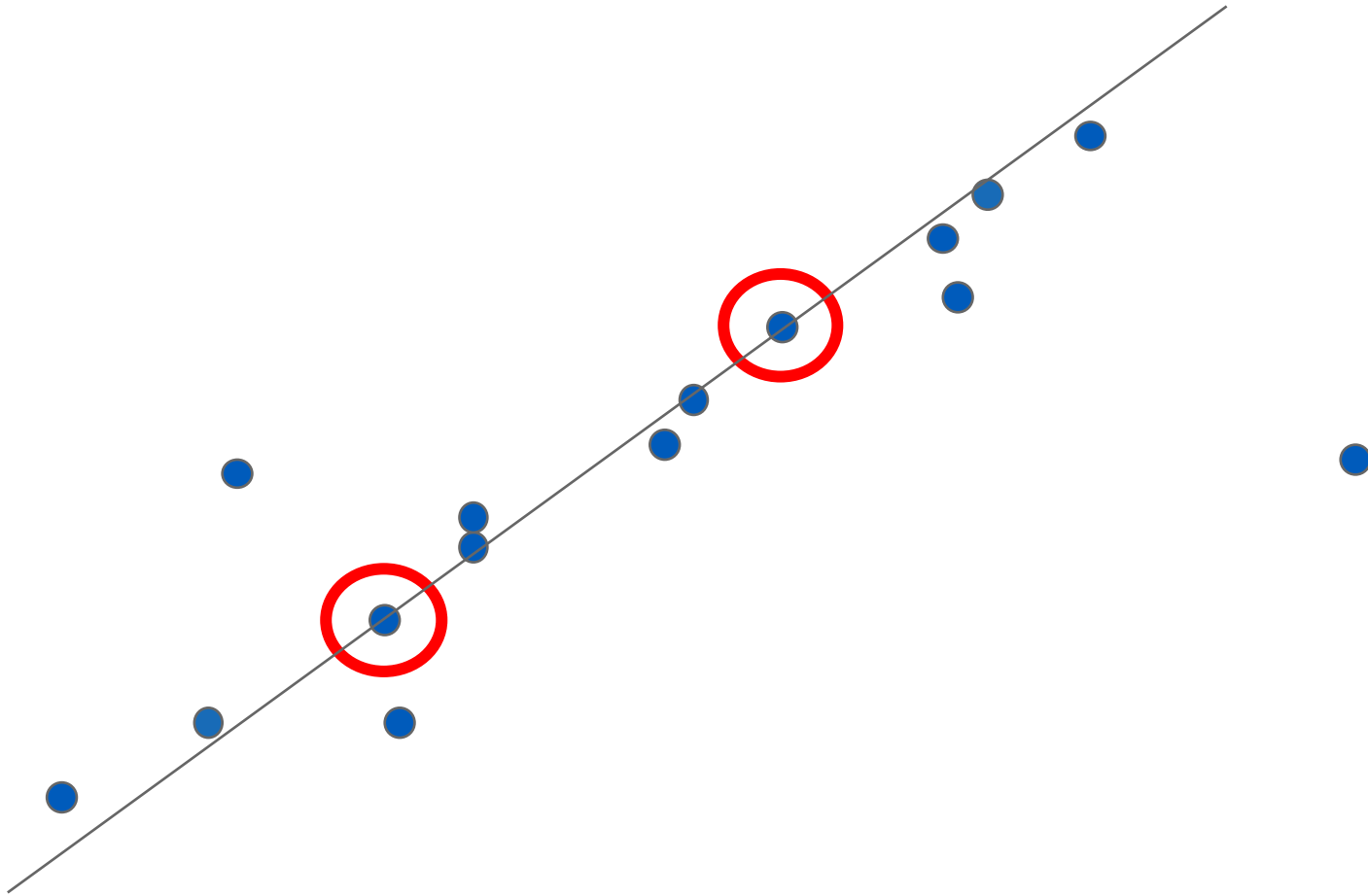
RANSAC Line Fitting Example

Repeat, until get a good result



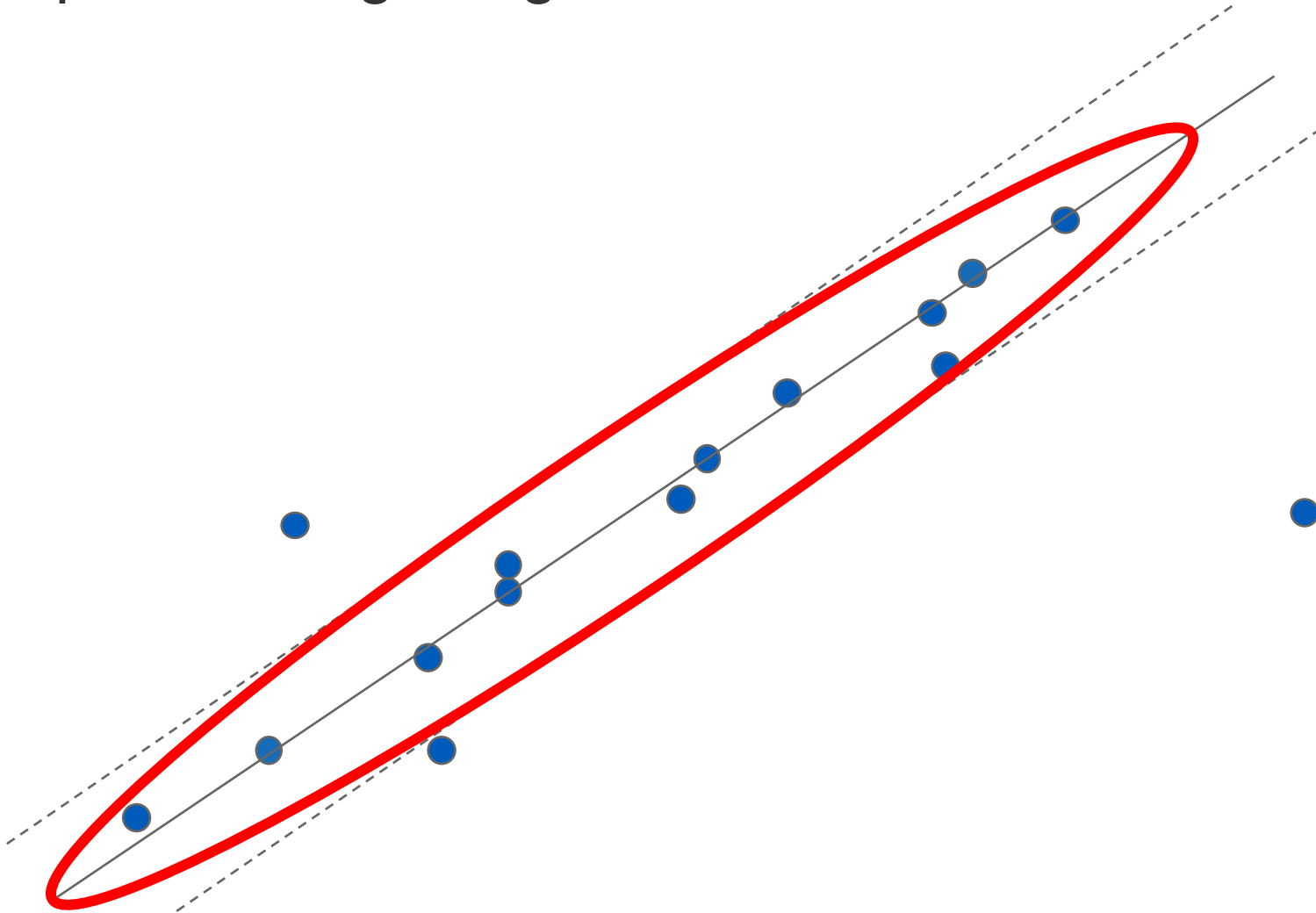
RANSAC Line Fitting Example

Repeat, until get a good result



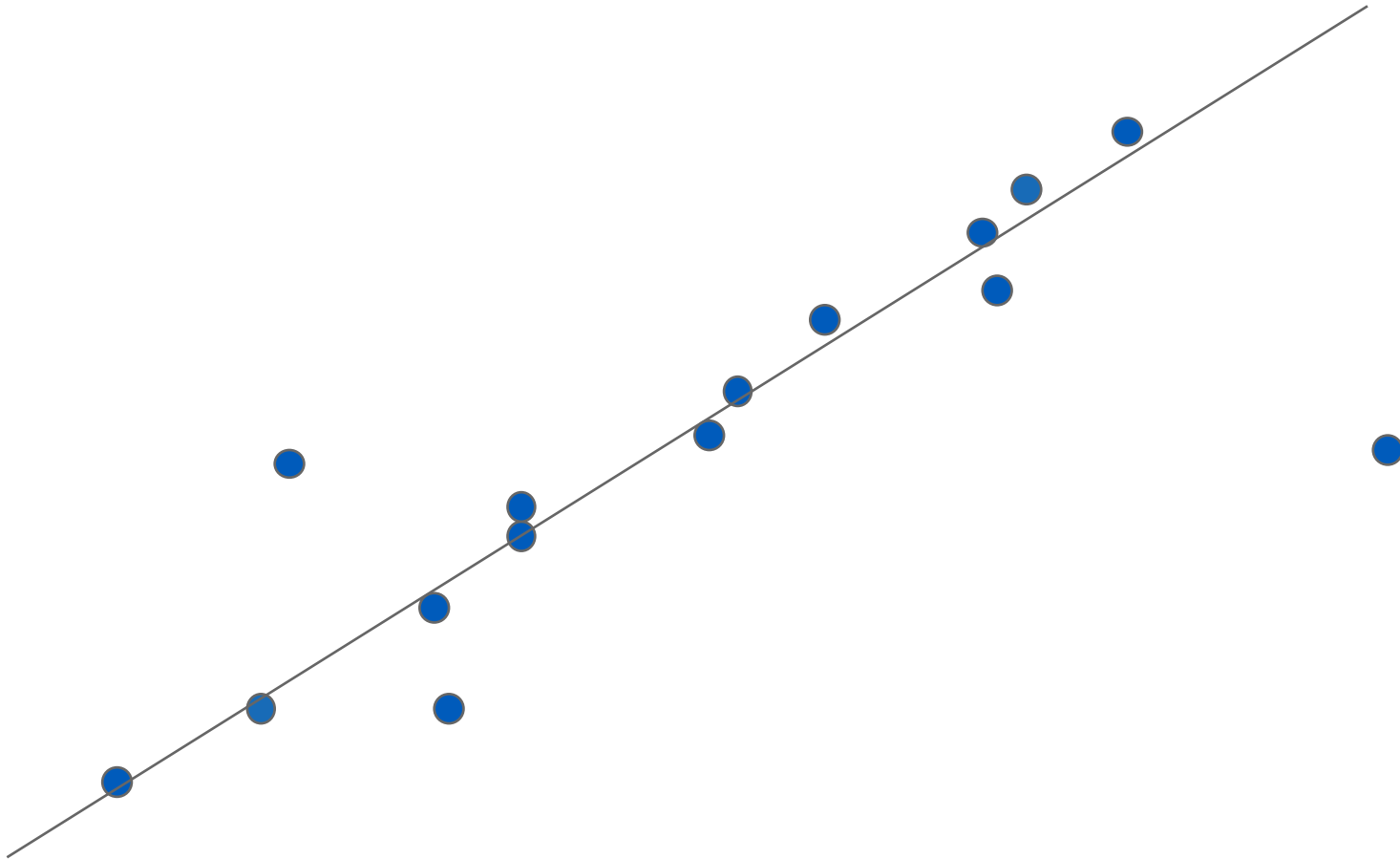
RANSAC Line Fitting Example

Repeat, until get a good result



RANSAC Line Fitting Example

Repeat, until get a good result

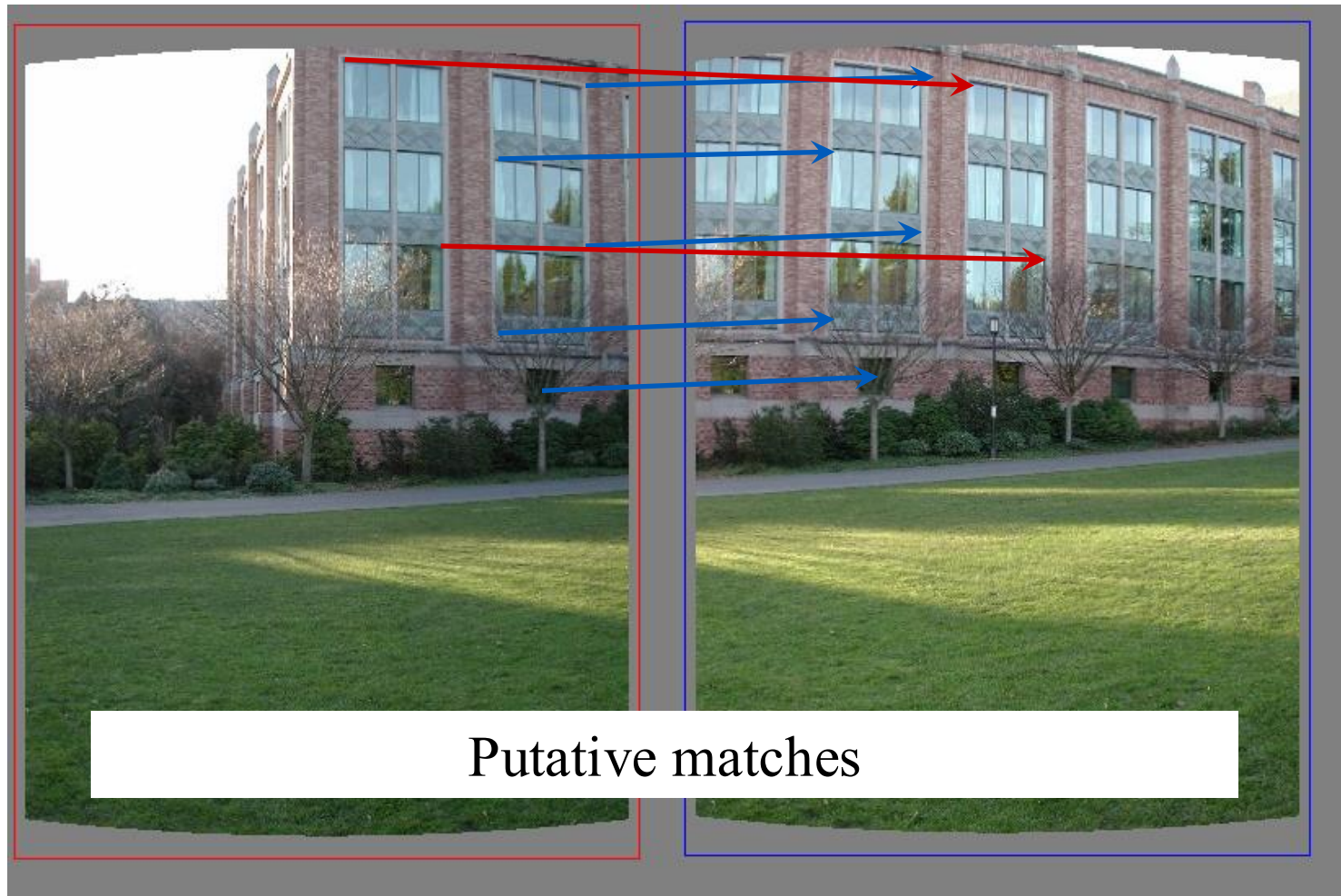


How to choose parameters?

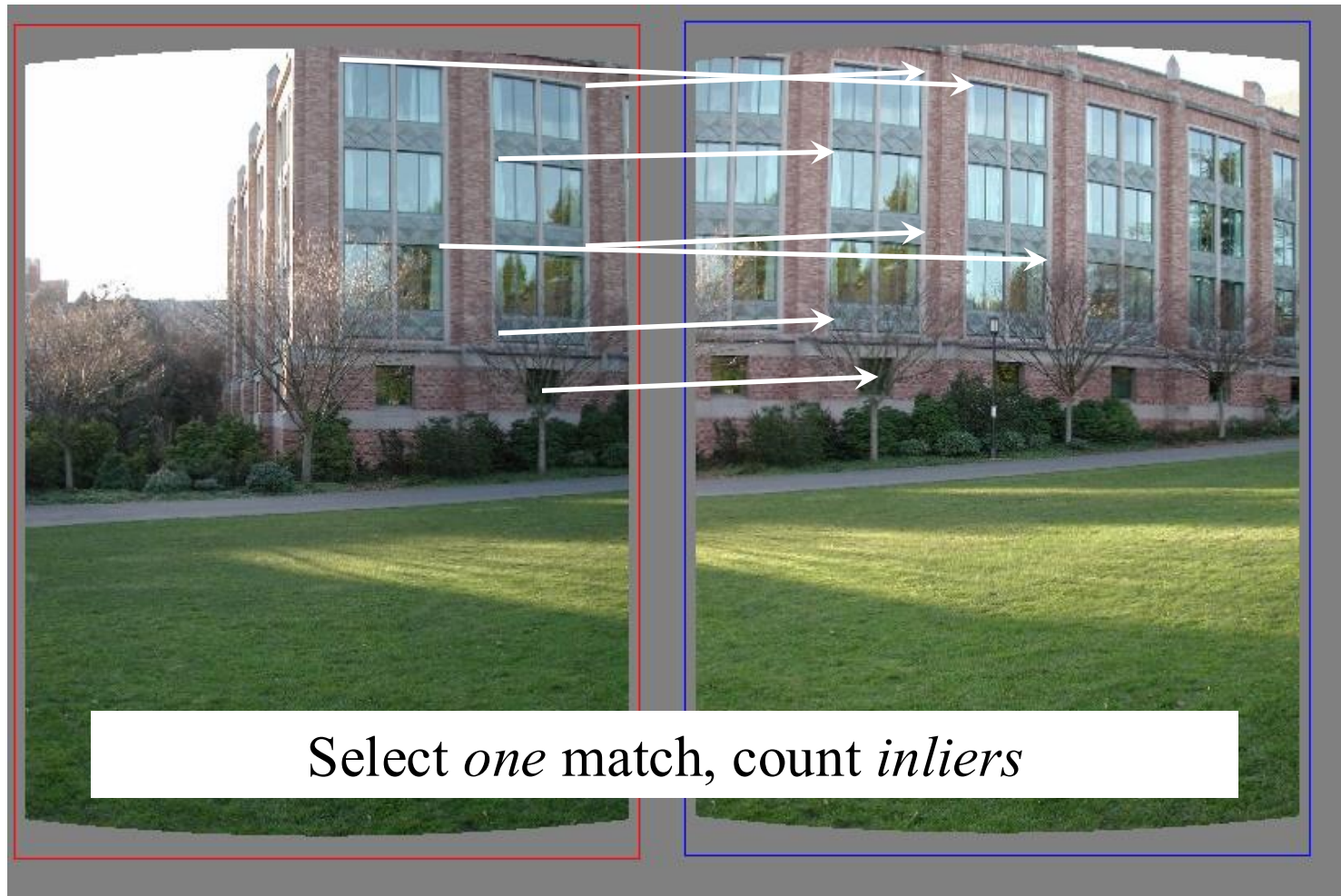
- Number of sampled points n : minimum points to fit a model.
- Inlier threshold δ .
 - Choose δ so that **a good point with noise** is likely within threshold.
- To determine the number of iterations K .
 - Desired probability of success (p): at least one useful result.
 - Let w be the probability of choosing an inlier when selecting a point.
 - w = number of inliers in data / number of points in data
 - n points selected independently for estimating a model.
 - w^n : the probability that all n points are inliers.
 - $1 - w^n$: probability of at least one of the n points is an outlier.
 - $(1 - w^n)^K$: after k iterations, never select a set of n inlier points.
 - $1 - p = (1 - w^n)^K$
 - $K = \frac{\log(1-p)}{\log(1-w^n)}$

n	$p = 0.99$, proportion of outliers ($1 - w$)						
	$1 - w = 5\%$	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

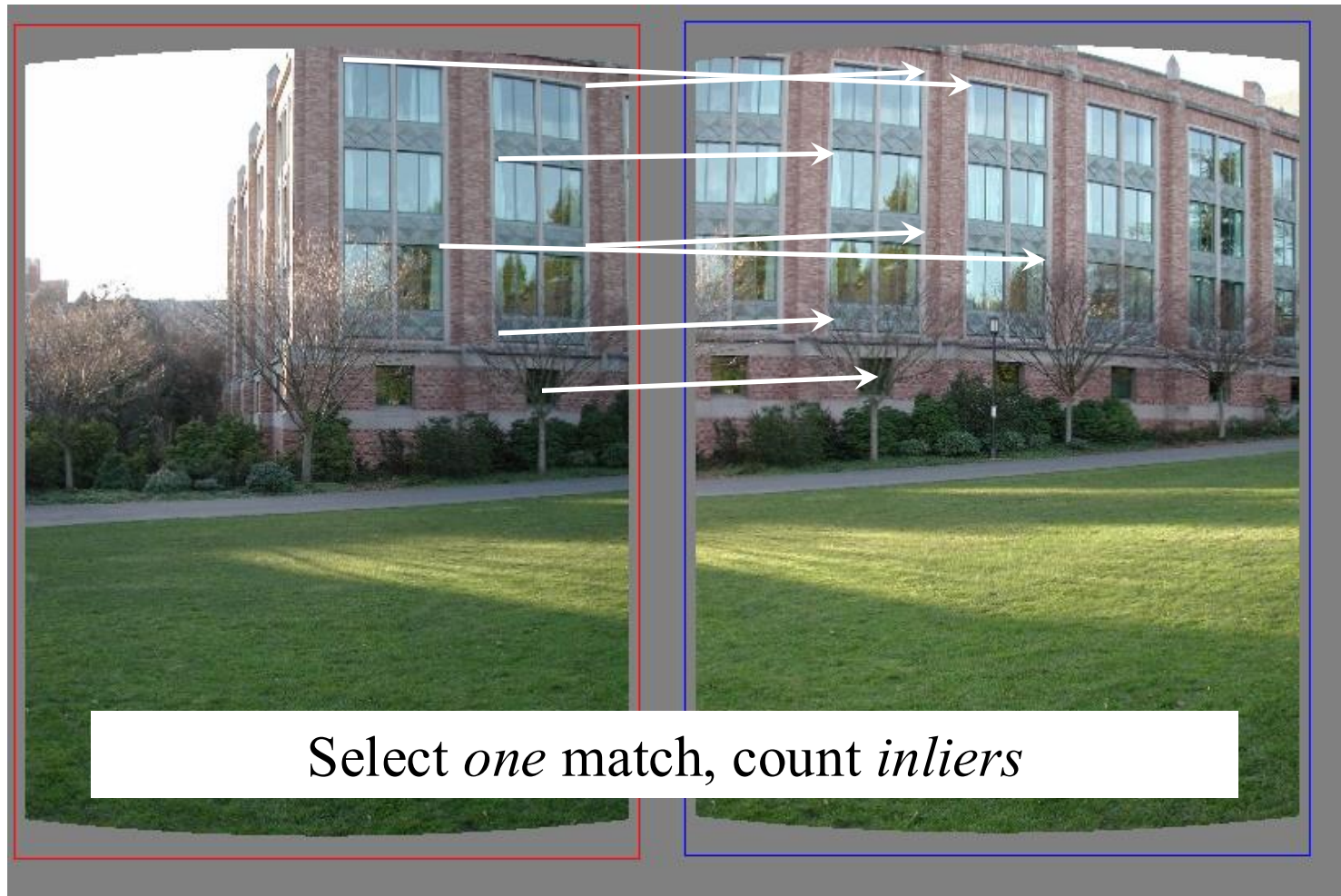
RANSAC example: Translation



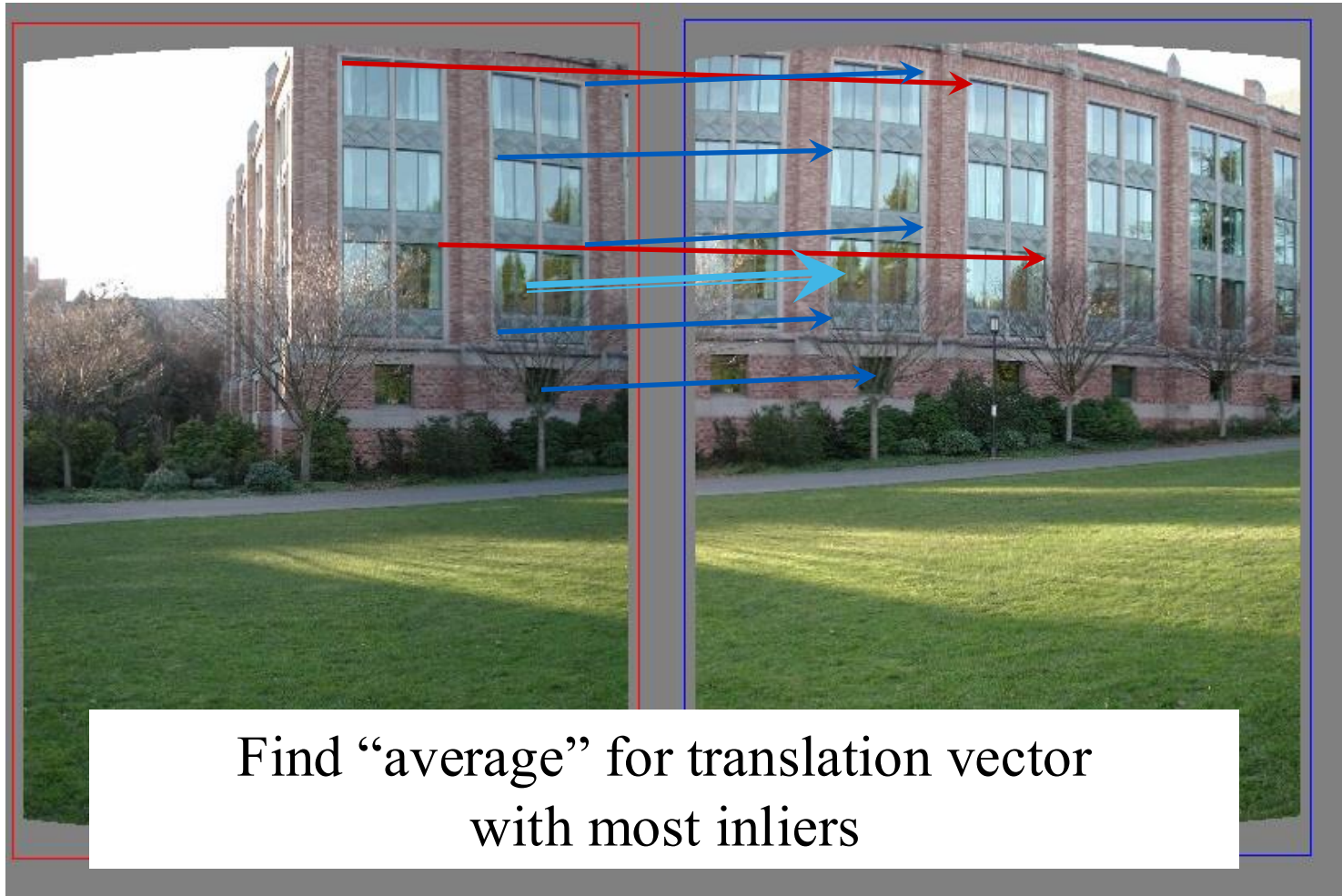
RANSAC example: Translation



RANSAC example: Translation

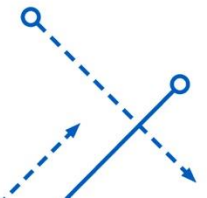


RANSAC example: Translation



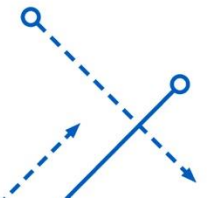
Summary

- Choose:
 - Inlier threshold
 - Related to the amount of noise we expect
 - Number of rounds
 - Related to the percentage of outliers we expect
 - Related to the probability of success we are hoping for.



RANSAC ALGORITHM

- Input:
 - Data: a set of observed data points
 - Model: a model to fit the data
 - Threshold: a threshold to determine inliers
- Output: BestModel: the model with the most inliers
- Repeat for a fixed number of iterations:
 - 1. Select a random subset of data
 - 2. Fit the model to the data points in the subset
 - 3. Determine the inliers by comparing the fitted model to data
 - 4. If the number of inliers exceeds the threshold
 - re-estimate the model using all the inliers
 - 5. Store the model if it has the most inliers seen so far
- Return BestModel



RANSAC Properties

Strength

- Robust to outliers
- Applicable for larger number of model parameters.

Weakness

- Computational time grows quickly with outliers and parameters
- Not good for getting multiple fits

Many applications

- Computing a homography (e.g., image stitching)
- Estimating fundamental matrix (relating two views, future lectures)

How to stitch together a panorama?

- Take a sequence of images from the same position
 - Rotate the camera about its optical center
- Compute transformation between second image and first.
 - Run RANSAC if needed.
- Transform the second image to overlap with the first
- Blend the two together to create a mosaic
- If there are more images, repeat